

به نام خدا

گزارش تمرین کامپیوتری سوم

سیگنال ها و سیستم ها

دکتر اخوان

ناژین نیکخواه بهرامی ۸۱۰۱۰۲۵۳۰

فروردین ۱۴۰۴

بخش اول:

تمرین ۱-۱)

در این قسمت با استفاده از حروف کوچک انگلیسی و علامت های گفته شده در مجموع ۳۲ کاراکتر داریم که میخواهیم هر کاراکتر را به ۵ بیت مرتبط کنیم. برای اینکار به صورت زیر با استفاده از سل یک مپ ست ۳۲*۲ ساختیم که ردیف اول شامل حروف و علامت ها است و در ردیف دوم با استفاده از راهنمایی سوال و استفاده از تابع مورد نظر یک عدد باینری را به آن مپ کرده ایم.

```
% P1-1 Creating a mapset
Nch=32;
mapset=cell(2,Nch);
Alphabet = 'abcdefghijklmnopqrstuvwxyz .,!";';
for i=1:Nch
    mapset{1,i}=Alphabet(i);
    mapset{2,i}=dec2bin(i-1,5);
end
```

تمرین ۱-۲)

ابتدا عکس مورد نظر خود را که قصد داریم پیام را در آن بگنجانیم، خوانده و سیاه و سفید میکنیم تا به همراهم پیام به ورودی تابع داده شود.

```
% p1-2
%Get the picture and make it b&w
X=imread('flower.jpg');
X=rgb2gray(X);
message='signal;';
Y=coding(message, X, mapset);
```

در تابع مورد ۳ ورودی داریم و یک خروجی. ورودی ها شامل پیام، مپ ست و عکس به صورت سیاه و سفید است که در قسمت قبل نوشته شد.

برای ورودی پیام هر حرف را تشخیص داده و معادل باینری را ذخیره میکند. سپس آرایه ای از اعداد باینری داریم که تبدیل به ماتریس خواهد شد.

```
function encoded_image = coding(message, X, mapset)

message_len=length(message);
message_bin=cell(1,message_len);

for i=1:message_len
    ch=message(i);
    index=strcmp(ch,mapset(1,:));
    message_bin{i}=mapset{2,index};
end

binarymessage=cell2mat(message_bin);
binarymessage_len=length(binarymessage);
```

بنابراین تا این مرحله دو تا از ورودی های تابع درست شده اند و مپ ست نیز برای تشخیص پیام آماده است.

در قسمت بعد احتیاج داریم بلاک هایی که پیام در آنها مخفی میشوند را پیدا کنیم. و با یک ویژگی و قسمتی از مختصات مکانی(گوشه راست یا چپ یا مثلا مرکز) آنها را ذخیره کنیم تا محل مناسب پیام یافت شود.

اندازه بلاک ها را باتوجه به راهنمایی سوال ۵*۵ در نظر میگیریم و برای ملاک دسته بندی و مرتب کردن بلاک های عکس، واریانس هر بلاک را در نظر میگیریم.

```
block_size = 5;
[sorted_coordinates, sorted_variances] = blocking_image(X, block_size);
```

داخل یک تابع این موضوع را بررسی میکنیم.

باتوجه به اندازه بلوک مورد نظر و هم چنین اندازه عکس، تعداد بلاک ها بدست می آید. بنابراین به ترتیب شروع کرده و از مقدار اولیه ۰ ذخیره سازی میکنیم.

```
function [sorted_coordinates, sorted_variances] = blocking_image(img, block_size)
```

```
[img_height, img_width] = size(img);  
num_blocks_y = img_height / block_size;  
num_blocks_x = img_width / block_size;  
variances = zeros(fix(num_blocks_y), fix(num_blocks_x));
```

حالا باتوجه به تعداد و اندازه مدنظرمان سطر و ستون بلاک ها را معلوم کرده و واریانس را بدست می آوریم (که یک ویژگی براساس تغییر رنگ بلاک برای مرتب سازی است) و ذخیره میشود.

```
for i = 1:num_blocks_y  
    for j = 1:num_blocks_x  
        row_start = (i-1) * block_size + 1;  
        row_end = i * block_size;  
        col_start = (j-1) * block_size + 1;  
        col_end = j * block_size;  
        block = img(row_start:row_end, col_start:col_end);  
  
        variances(i, j) = var(double(block(:)));  
    end  
end
```

حالا آرایه ای که داخلش مقادیر ذخیره شده است را باید مرتب کنیم. به همین منظور ابتدا لیست تهیه میکنیم و بعد آن را سورت میکنیم.

```
variances_flat = variances(:);  
[sorted_variances, sorted_indices] = sort(variances_flat, 'descend');
```

حالا برای ذخیره سازی مکان معلوم شده مختصات گوشه هر بلاک را بر اساس واریانس مرتب و ذخیره میکنیم. (براساس کم اهمیت تا پر اهمیت)

با توجه با شماره بلاک در لیست واریانس و تعداد کل بلاک هایی که واریانس برای آنها محاسبه شده است، سطر و ستون را پیدا میکند و میتوانیم مختصات گوشه مدنظر را برای آنها بیابیم.

```
sorted_coordinates = zeros(length(sorted_indices), 2);  
for k = 1:length(sorted_indices)  
    [row, col] = ind2sub(size(variances), sorted_indices(k));  
    sorted_coordinates(k, :) = [(col-1) * block_size + 1, (row-1) * block_size + 1];  
end  
end
```

حالا خروجی این تابع میتواند وارد تابع تمرین ۱-۲ شود و محل قرار گرفتن پیام را معلوم کند.

```
encoded_image = X;
threshold = 0.5;
message_index = 1;
```

پیام را به ترتیب بیشترین تغییر به کمترین تغییر رنگ یا همین واریانس کدگذاری میکنیم.

برای اینکار احتیاج به یک ترشهولد داریم یعنی مرزی که بیان کند کمترین شلوغی برای محل پنهان شدن پیام باید چقدر باشد و اگر واریانس از این عدد کمتر بود، کلا آن بلاک مدنظر ما نیست.

حالا باید به نحوی بلاک ها را جدا کنیم و بعد داخل آنها مراحل را پیش ببریم.

به کمک خط و ستون ها گوچه سمت چپ را جدا کرده و میتوانیم بلاک ها را از روی عکس جدا کنیم.

(By using sorted_cordinates)

باتوجه به صورت سوال:

بگنجانیم. ابتدا حروف پیام را به صورت باینری در آورید که (با توجه به جدول بالا) می شود: 000010001011111
، سپس رشته باینری به دست آمده را مطابق آنچه در کلاس توضیح داده شد در پیکسل های بلوک های منتخب تصویر بگنجانید (هر بیت باینری را در کم اهمیت ترین بیت هر پیکسل قرار می دهیم). توجه داشته باشید پیام ممکن است هر کلمه یا جمله ای با طول دلخواه شامل ۳۲ کاراکتر ذکر شده باشد. همچنین اگر طول پیام باینری شده از تعداد پیکسل های بلوک های منتخب تصویر بیشتر بود، تابع شما به کارب خطا دهد.

حالا باز هم میخواهیم در بین شلوغ ترین بلاک عکس کمترین تغییر را داشته باشیم.

بنابراین در عکس مورد نظر (که ابتدا سیاه و سفید شده بود) باید رشته باینری را در پیکسل های آن پنهان کنیم.

به این صورت که پیکسل اول را باید با گری اسکیل تبدیل به یک عدد رنگ میکنیم بعد به باینری تبدیل کنیم. رشته پیام باینری را نیز بدست آوریم و بیت آخر پیام را با بیت آخر خانه (کم اهمیت ترین از نظر مقداری) جا به جا کنیم و به همین ترتیب آنقدر جلو میرویم که پیام تمام ود یا بیت های خانه بلاک کنونی. اگر پیام تمام شد که کار تمام شده است و در غیر این اگر احتیاج به "جا" برای گنجاندن داشتیم به بلاک

بعدی با بیشترین واریانس میرویم.(خلاصه این کار: ۱-ترشهولد، ۲-همه چیز ۰ و ۱، ۳-جایگزینی با کم اهمیت ترین)

اما همانطور که گفته شده است، اگر تعداد بیت پیام بیشتر بود بتواند از حلقه خارج شود و ارور دهد. بنابراین احتیاج به یک ترشهولد دیگر برای تعداد بیت خواهیم داشت.

```
for k = 1:length(sorted_variances)
    try
        if(sorted_variances(k) < threshold)
            error("message length is bigger than pixel numbers")
        end
    catch ME
        exit
    end

    coords = sorted_coordinates(k, :);
    row_start = coords(2);
    col_start = coords(1);

    block = encoded_image(row_start:row_start+4, col_start:col_start+4);

    for i = 1:block_size %چرخیدن بلاک به بلاک
        for j = 1:block_size
            if message_index <= binarymessage_len
                pixel_value = block(i, j);
                pixel_bin = dec2bin(pixel_value);
                pixel_bin(end) = binarymessage(message_index);
                block(i, j) = bin2dec(pixel_bin);
                message_index = message_index + 1;
            else
                break;
            end
        end
    end

    encoded_image(row_start:row_start+4, col_start:col_start+4) = block;
    if message_index > binarymessage_len
        break;
    end
end
end
```

بیت به بیت پیام را در یلاک قرار میدهد. هر سری چک میکنیم که اگر به انتهای پیام نرسیدیم جلو برویم و به پیکسل بعدی منتقل شود. اگر بلاک فعلی دیگر جا نداشته باشد ولی هنوز پیغام ادامه داشته باشد، به بلاک بعدی طبق ترتیب گفته شده در بالا میرود.

تمرین ۱-۳)

بعلت اینکه بیت های پیام در شلوغ ترین قسمت های عکس، بیت های پیام با کم اهمیت ترین پیکسل های بلاک های عکس جا به جا شده است، عکس اصلی و کد شده با هم یکی اند.

```
%% p1-3
message='signal;';
Y=coding(message, X, mapset);
subplot(1,2,1)
imshow(X)
title('Original picture')
subplot(1,2,2)
imshow(Y)
title('Coded picture')
```

Original picture



Coded picture



تمرین ۱-۴)

در این بخش قصد یافت کردن کد درون عکس را داریم. بنابراین ورودی ها پیام کدگذاری شده، مپ ست و استراتژی انتخاب بلوک مثل ترشهولد و ابعاد باشد و در خروجی پیام باشد که از عکس بدست آمده است.

```
%% p1-4
threshold = 0.5;
block_size = 5;
decoded_message = decoding(Y, mapset, threshold, block_size);
disp(decoded_message)
```

انتظار داریم که همان ورودی که در قسمت ۳-۱ داده بودیم را ببینیم و خروجی:

```
Command Window
signal;
fx >>
```

همانطور که مشاهده میشود احتیاج به تابعی داریم به اسم Decoding.

ابتدا به کمک مپ ست لیست حروف را میسازیم تا بتوان حروف یا کاراکتر های پنهان شده را پیدا کرد. بعد از آن احتیاج داریم مثل بخش کودینگ، یک بلاک کردن انجام دهیم و هر قسمت را بررسی کنیم. و هر قسمت را برحسب میزان شلوغی رنگ ها یعنی واریانس مرتب کنیم زیرا در قسمت های شلوغ احتمال پنهان شدن حرف یا کاراکتر بیشتر است.

اما در اینجا عکس عمل کدینگ ما نمیدانیم که پیام چند بیت بوده یا تا چند بلاک ادامه داده شده است. پس باید یک متغیر به ما بگوید کجا پیام به اتمام رسیده است. و متغیر ما فلگ است.

```
function DcodedMessageBin = decoding(X, mapset, threshold, block_size)
    % خروجی ها و ورودی های تابع
    Alphabet = strjoin(mapset(1, :), '');
    % مپ ست
    [sorted_coordinates, sorted_variances] = blocking_image(X, block_size);
    % مرتب سازی با بلاک کردن و واریانس
    flag=1; % پیام کی تمام میشود؟
    DcodedMessageBin=[]; % کاراکتر استخراج شده;
```

حالا عکس کاری که در قسمت قبل کردیم یعنی از واریانس زیاد شروع میکنیم و بعد باتوجه به سطر و ستون آن و گوشه چپ بالا آن بلاک را جدا میکنیم.

سپس مقدار هر پیکسل را باینری کرده و بیت آخر را به عنوان بیت پیام انتخاب و ذخیره میکنیم. (در قسمت قبلی بیت پیام را با بیت پیکسل جا به جا میکنیم)

چون بلاک کردن ما 5×5 بود و هر حرف نیز با رشته ۵ تایی از بیت ها نمایش داده میشود، بعد از خواندن هر خط آن حرف را ذخیره میکنیم. (بعلت این نوع دسته بندی)

همانطور که در سوال نیز به آن اشاره شده است، هر زمانی به بررسییم پیام تمام شده و تمام کاراکتر ها را استخراج کرده ایم. (شرط توقف)

```
while flag
    message_index = 1;
    for k = 1:length(sorted_variances)
        if(sorted_variances(k) < threshold) اگر طول از یک میزانی کمتر باشد قابل قبول نیست
            flag = 0;
            break
        end
        9ords = sorted_coordinates(k, :);
        row_start = coords(2);
        col_start = coords(1);

        block = X(row_start:row_start+(block_size-1),
col_start:col_start+(block_size-1));
        % مراحل تبدیل پیکسل و پیام
        for i = 1:block_size
            characterbin=zeros(1,block_size);
            for j = 1:block_size
                pixel_value = block(i, j);
                pixel_bin = dec2bin(pixel_value);
                characterbin(j)=str2double(pixel_bin(end));
                message_index = message_index + 1;
            end
            num=sum(characterbin.*(2.^(4:-1:0)))+1;
            DcodedMessageBin=[DcodedMessageBin Alphabet(num)];
            if strcmp(Alphabet(num), ';') توقف
                flag=0;
            end
            if flag == 0
                break;
            end
        end
        if flag == 0
            break;
        end
    end
end
end
```

تمرین ۱-۵)

چون بعد از کدگذاری نویز وارد شده است بنابراین در وارد کردن کد خدشه ای وارد نشده است اما هنگام خواندن آن ممکن است مشکل ایجاد شود.

اگر نویز به بلاک هایی از عکس که واریانس کمتری دارد وارد شود در استخراج کد مشکلی ایجاد نمیشود.

اما اگر در ناحیه هایی با واریانس بیشتر که امکان پنهان شدن پیام در آنها بیشتر است، نویز وارد شود در فرآیند خواندن بیت ها ممکن است اختلال ایجاد شود و پیام:

۱-کامل خوانده نشود ۲-عوض شود ۳-نتواند توقف کند و خروجی بدهد و ...

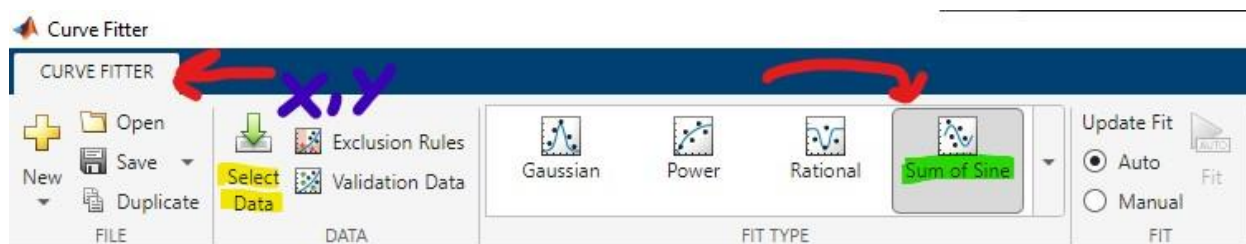
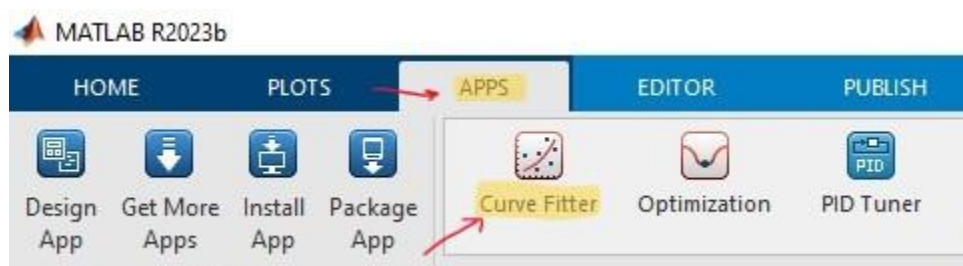
بخش دوم:

تمرین ۱-۲) ابتدا داده ها را لود میکنیم و میخوانیم:

```
% p4  
clc,clear,close all;  
%% p4-1 loading x & y data  
load DataFit.mat
```

Workspace	
Name	Value
x	1x1000 double
y	1x1000 double

از قسمت برنامه ها، برنامه مورد نظر را باز میکنیم و نمودار سینوسی را انتخاب میکنیم:

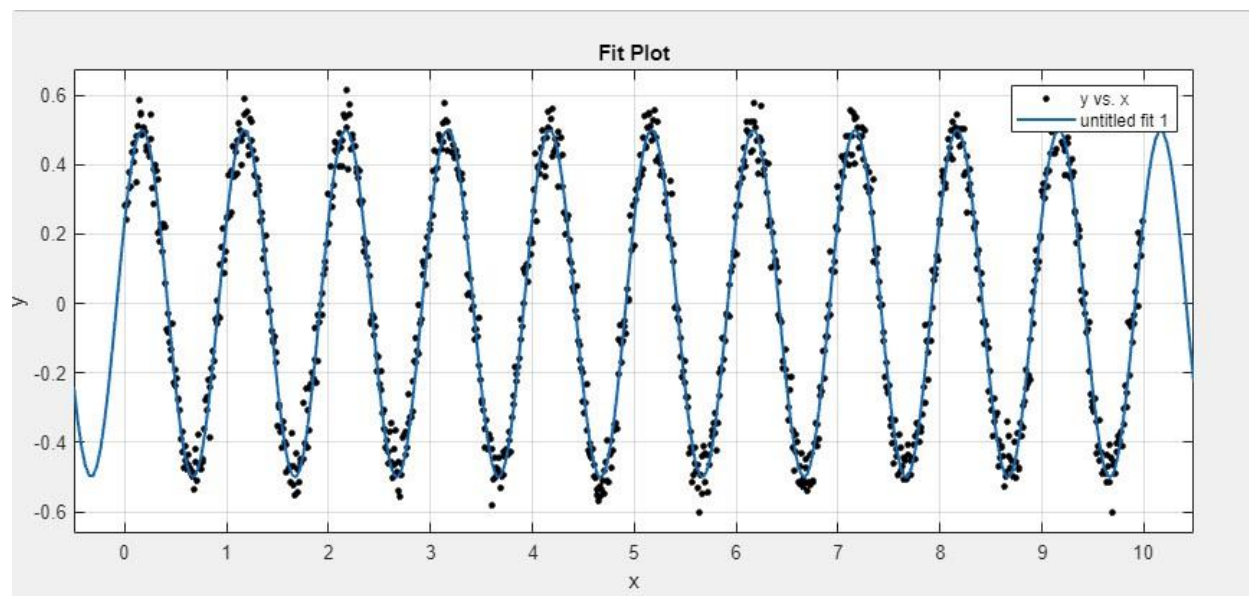


The image shows the 'Select Fitting Data' dialog box. It contains the following fields:

- Fit name: untitled fit 1
- X data: x
- Y data: y
- Z data: select
- Weights: select

خروجی ها به صورت زیر است و پارامتر ها به ترتیب:

1) $a_1 = \alpha_0$, 2) $b_1 = \omega_0$, 3) $c_1 = \phi_0$



▼ Results

Fit Name: untitled fit 1

Sum of Sine Curve Fit (sin1)
 $f(x) = a_1 \sin(b_1 x + c_1)$

Coefficients and 95% Confidence Bounds

	Value	Lower	Upper
a1	0.4990	0.4946	0.5034
b1	6.2858	6.2828	6.2889
c1	0.5037	0.4860	0.5214

Goodness of Fit

◀ [Progress Bar] ▶

	Value
a1	0.4990
b1	6.2858
c1	0.5037

تمرین ۲-۲) به صورت exhaustive search (یا جستجو به صورت فراگیر)،

به کمک کمینه سازی تابع هدف زیر می‌خواهیم سه پارامتر را استخراج کنیم:

$$f(a_0, \omega_0, \varphi_0) = \sum_{i=1}^{1000} (y_i - a_0 \sin(\omega_0 x_i + \varphi_0))^2$$

به طوری که ۳ حلقه فور، هر حلقه برای یک پارامتر، بنویسیم. همچنین به کمک دستور تیک تاک زمان این پروسه را نیز بیان می‌کنیم.

بازه ها را به صورت زیر در نظر می‌گیریم که در دستور کار آورده شده است:

$$\text{for } a_0 = 0.01 : 0.01 : 1$$

$$\text{for } \omega_0 = 0 : \frac{\pi}{10} : 10\pi$$

$$\text{for } \varphi_0 = 0 : \frac{\pi}{100} : 2\pi$$

- pseudo code کامل نوشته نشده و صرفاً برای این که به شما ایده بدهد به صورت کلی نوشته شده است. به

عنوان مثال متغیرهای m و n و p را می‌بایست در جای مناسب یکی یکی افزایش دهید تا درایه‌های بردار سه بعدی تابع هدف پر شود.

```

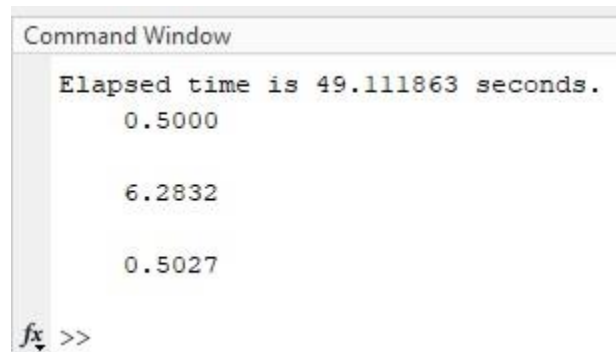
%% p4-2
tic
min_error= inf;

a0= 0; w0= 0; phi0= 0;

for a=0.01:0.01:1
    for w=0:pi/10:10*pi
        for phi=0:pi/100:2*pi
            error= sum((y-a*sin(w*x+phi)).^2); مجموع مربع فاصله است
            if error < min_error
                min_error= error;
                a0= a; w0= w; phi0= phi;
            end
        end
    end
end
toc
disp(a0);
disp(w0);
disp(phi0);

```

در بازه های معلوم شده سه متغیر تغییر میکنند تا با فرمول گفته شده، هر سری یک خطا محاسبه شود و با مرحله قبل مقایسه شود حالا اگر این مقدار خطا به ازای این ۳ متغیر کمتر از قبل بود این مقدار ذخیره شده و سه تا متغیر را به عنوان هدف نگه میداریم و این روند تا آخر بازه ادامه پیدا میکند و در نهایت کمترین پیدا میشود. (شرط توقفی نداریم) توجه کنید که ۳ تا حلقه باید تو در تو باشد تا هر سه تا با هم در یک مرحله داخل روند محاسبه باشند.



```

Command Window

Elapsed time is 49.111863 seconds.
    0.5000

    6.2832

    0.5027

fx >>

```

خروجی به این صورت است و همانطور که مشخص است بعلت طولانی بودن بازه و قدم های جلو رفتن ۳ تا حلقه زمان زیادی طول میکشد تا خروجی پیدا کنیم. (۱۰۰ در ۱۰۰ در ۲۰۰ بار)

تمرین ۲-۳

```

%% p2-3
tic
min_error= inf;

a0= 0; w0= 0; phi0= 0;

for a=0.1:0.1:1
    for w=0:pi:10*pi
        for phi=0:pi/10:2*pi

```

```

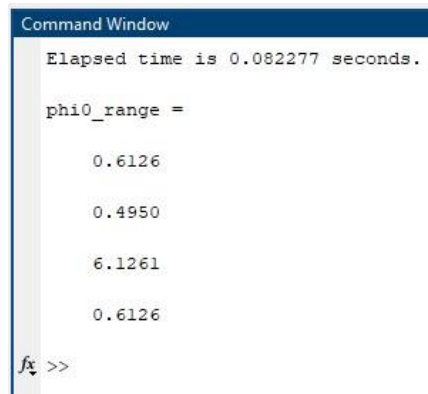
        error= sum((y-a*sin(w*x+phi)).^2);
        if error < min_error
            min_error= error;
            a0= a; w0= w; phi0= phi;
        end
    end
end
toc
min_error= inf;
a0_range= max(a0-0.005,0.01):0.01:min(a0+0.005,1);
w0_range= max(w0-pi/20,0):pi/10:min(w0+pi/20,10*pi);
phi0_range= max(phi0-pi/200,0):pi/100:min(phi0-pi/200,2*pi)
a0= 0; w0= 0; phi0= 0;

for a=a0_range
    for w=w0_range
        for phi=phi0_range
            error= sum((y-a*sin(w*x+phi)).^2);
            if error < min_error
                min_error= error;
                a0= a; w0= w; phi0= phi;
            end
        end
    end
end

disp(a0);
disp(w0);
disp(phi0);

```

در این قسمت با همان الگوریتم و مرحله ها سعی میکنیم زمان را کمتر کنیم و نتیجه مطلوب را بدست بیاوریم. به جای اینکه ۱۰۰ تایی مثلا جلو برویم و همه قسمت ها را بررسی کنیم، انگار در یک قسمت بزرگ تر بررسی میکنیم و ببینیم به چه مقداری میرسیم و این سه تا مقدار نزدیک اند. حالا دوباره در بازه تعیین شده قدم های کوچک برمیداریم و بررسی میکنیم. (بازه بزرگ قدم های بزرگ و بازه کوچک قدم کوچک به جای بازه بزرگ و قدم کوچک) بنابراین خروجی این قسمت سریع تر به همگرایی میرسد.



```

Command Window

Elapsed time is 0.082277 seconds.

phi0_range =

    0.6126
    0.4950
    6.1261
    0.6126

fx >>

```

تمرین ۲-۴)

$$z^{(k)} = \begin{bmatrix} a_0^{(k)} \\ \omega_0^{(k)} \\ \phi_0^{(k)} \end{bmatrix}; \quad z^{(k+1)} \leftarrow z^{(k)} - \mu \nabla f_{z^{(k)}}$$

بردار گرادین در نقطه $z^{(k)}$ یک عدد است (مثلاً $\mu=0.01$)

```
% p2-4
tic
mu= 0.000001;
tol= 0.00001;
z= [0.5;6;0.5]; عدد نزدیک می‌دهیم که همگرا شود;
z_new = [inf,inf,inf];
grad = [inf,inf,inf];
for i= 1:100000
    a= z(1); w= z(2); phi= z(3);
    grad= [-2*sum((y-a.*sin(w.*x+phi)).*sin(w.*x+phi));
           -2*sum((y-a.*sin(w.*x+phi)).*a.*x.*cos(w.*x+phi));
           -2*sum((y-a.*sin(w.*x+phi)).*a.*cos(w.*x+phi))];
    z_new= z-mu*grad;
    if norm(z_new-z) < tol || norm(grad) < tol
        break
    end
    z = z_new;
end
toc
disp(z);
```

در این قسمت هر دفعه گرادین (مشتق بر حسب ۳ متغیر) را طبق راهنمایی گفته شده بدست می آوریم و تئرانس یا فاصله را با اندازه مقایسه میکنیم تا دوباره به کمترین مقدار برسیم و تابع هدف و متغیر ها بدست بیایند. (مقدار ضریب باید مناسب (خیلی کوچک) باشد و گرادین را درست بدست بیاید تا به عدد درست همگرا شود و بینهایت شود)

```
Command Window

Elapsed time is 0.428339 seconds.
0.4890
6.2802
0.5413

fx >>
```


تمرین ۲-۵)

در این قسمت از همان کد قبل استفاده میکنیم و خروجی ها را بدست می آوریم و میبینیم که تابع مینیمم گلوبال ندارد.

```
z= [1;6;4];
```

Elapsed time is 12.582815 seconds.

0.4888

6.2302

0.8736

```
z= [0.2;8;0.3];
```

Elapsed time is 0.527714 seconds.

-0.0065

7.9792

0.2936

```
z= [50;20;30];
```

Elapsed time is 1.287304 seconds.

0.0183

19.8934

30.7475

خروجی ها نادرست اند و شباهتی ندارد.(اگر حلقه فور نبود با ازای ورودی بزرگ ادامه دارد)

بخش سوم:

IMPORT

VIEW

Delimited

Column delimiters:

Comma

Fixed Width

Delimiter Options

DELIMITERS

Range:

A2:G601

Output Type:

Table

Variable Names Row:

1

SELECTION

Replace

unimportable cells with

NaN

UNIMPORTABLE CELLS

Import Selection

The following variables were imported:
diabetestraining.(600x7)

diabetes-training.csv

	A	B	C	D	E	F	G
	diabetestraining						
	Glucose	BloodPress...	SkinThickn...	Insulin	BMI	Age	label
	Number	Number	Number	Number	Number	Number	Number
1	Glucose	BloodPress...	SkinThickn...	Insulin	BMI	Age	label
2	148	72	35	0	33.6	50	1
3	85	66	29	0	26.6	31	0
4	183	64	0	0	23.3	32	1
5	89	66	23	94	28.1	21	0
6	137	40	35	168	43.1	33	1
7	116	74	0	0	25.6	30	0
8	78	50	32	88	31	26	1
9	115	0	0	0	35.3	29	0
10	197	70	45	543	30.5	53	1
11	125	96	0	0	0	54	1
12	110	92	0	0	37.6	30	0
13	168	74	0	0	38	34	1
14	139	80	0	0	27.1	57	0
15	189	60	23	846	30.1	59	1
16	166	72	19	175	25.8	51	1
17	100	0	0	0	30	32	1
18	118	84	47	230	45.8	31	1
19	107	74	0	0	29.6	31	1
20	103	30	38	83	43.3	33	0
21	115	70	30	96	34.6	32	1
22	126	88	41	235	39.3	27	0
23	99	84	0	0	35.4	50	0

که مشاهده می کنید این دیتاست شامل شش اطلاعات و علامت پزشکی (یا فیچر) اندازه گیری شده از ۶۰۰ نفر مختلف است. ستون آخر نشان دهنده ی این است که فرد دیابت دارد (برچسب ۱) و یا خیر (برچسب ۰).

می خواهیم به یک ماشین آموزش دهیم که چگونه با استفاده از شش ویژگی استخراج شده از هر فرد تصمیم بگیرد که فرد مورد نظر دیابت دارد یا خیر.

Models

Sort by Model Number

↓

↑

📊

📄

🗑️

☆ 1 Tree	Accuracy (Validation): 72.3%
Last change: Fine Tree 6/6 features	
☆ 2 SVM	Accuracy (Validation): 77.3%
Last change: Linear SVM 6/6 features	
☆ 3 SVM	Accuracy (Validation): 74.3%
Last change: Removed 5 features 1/6 features	
☆ 4 SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features	
☆ 5 SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features	
☆ 6 SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features	
☆ 7 SVM	Accuracy (Validation): 65.5%
Last change: Removed 5 features 1/6 features	
☆ 8 SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features	

Model 1

Model 2

Model 3

Model 4

M

Summary ×

Validation Confusion Matrix ×

Model 2: SVM

Status: Trained

Training Results

Accuracy (Validation)

77.3%

Total cost (Validation)

136

Prediction speed

~1500 obs/sec

Training time

21.893 sec

Model size (Compact)

~25 kB

Model Hyperparameters

Feature Selection: 6/6 individual features selected

PCA: Disabled

Misclassification Costs: Default

Optimizer: Not applicable

تمرین ۲-۳)

با استفاده از داپلیکیت کردن داریم:

۱-گلوکز

Models

Sort by Model Number

1 Tree Accuracy (Validation): 72.3% Last change: Fine Tree 6/6 features

2 SVM Accuracy (Validation): 77.3% Last change: Linear SVM 6/6 features

3 SVM Accuracy (Validation): 74.3% Last change: Removed 5 features 1/6 features

4 SVM Accuracy (Validation): 65.3% Last change: Removed 5 features 1/6 features

5 SVM Accuracy (Validation): 65.3% Last change: Removed 5 features 1/6 features

6 SVM Accuracy (Validation): 65.3% Last change: Removed 5 features 1/6 features

7 SVM Accuracy (Validation): 65.5% Last change: Removed 5 features 1/6 features

8 SVM Accuracy (Validation): 65.3% Last change: Removed 5 features 1/6 features

Model 1 Model 2 Model 3 Model 4 M

Summary Validation Confusion Matrix

Model 3: SVM Status: Trained

Training Results

Accuracy (Validation) 74.3% Total cost (Validation) 154 Prediction speed ~12000 obs/sec Training time 24.439 sec Model size (Compact) ~12 kB

Model Hyperparameters

Feature Selection: 1/6 individual features selected

PCA: Disabled

Misclassification Costs: Default

Optimizer: Not applicable

Models

Sort by Model Number

1	Tree	Accuracy (Validation): 72.3%
Last change: Fine Tree 6/6 features		
2	SVM	Accuracy (Validation): 77.3%
Last change: Linear SVM 6/6 features		
3	SVM	Accuracy (Validation): 74.3%
Last change: Removed 5 features 1/6 features		
4	SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features		
5	SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features		
6	SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features		
7	SVM	Accuracy (Validation): 65.5%
Last change: Removed 5 features 1/6 features		
8	SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features		

Model 1 Model 2 Model 3 Model 4

Summary Validation Confusion Matrix

Model 4: SVM
 Status: Trained

Training Results
 Accuracy (Validation) 65.3%
 Total cost (Validation) 208
 Prediction speed ~15000 obs/sec
 Training time 3.0379 sec
 Model size (Compact) ~14 kB

▶ **Model Hyperparameters**
 ▶ **Feature Selection: 1/6 individual features selected**
 ▶ **PCA: Disabled**
 ▶ **Misclassification Costs: Default**
 ▶ **Optimizer: Not applicable**

Models

Sort by Model Number

1	Tree	Accuracy (Validation): 72.3%
Last change: Fine Tree 6/6 features		
2	SVM	Accuracy (Validation): 77.3%
Last change: Linear SVM 6/6 features		
3	SVM	Accuracy (Validation): 74.3%
Last change: Removed 5 features 1/6 features		
4	SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features		
5	SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features		
6	SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features		
7	SVM	Accuracy (Validation): 65.5%
Last change: Removed 5 features 1/6 features		
8	SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features		

Model 1 Model 2 Model 3 Model 4 M

Summary x Validation Confusion Matrix x

Model 5: SVM
 Status: Trained

Training Results
 Accuracy (Validation) 65.3%
 Total cost (Validation) 208
 Prediction speed ~11000 obs/sec
 Training time 2.8702 sec
 Model size (Compact) ~14 kB

▶ **Model Hyperparameters**
 ▶ **Feature Selection: 1/6 individual features selected**
 ▶ **PCA: Disabled**
 ▶ **Misclassification Costs: Default**
 ▶ **Optimizer: Not applicable**

Models

Sort by Model Number

↓

↑

📄

📋

🗑️

☆ 1 Tree	Accuracy (Validation): 72.3%
Last change: Fine Tree 6/6 features	
☆ 2 SVM	Accuracy (Validation): 77.3%
Last change: Linear SVM 6/6 features	
☆ 3 SVM	Accuracy (Validation): 74.3%
Last change: Removed 5 features 1/6 features	
☆ 4 SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features	
☆ 5 SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features	
☆ 6 SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features	
☆ 7 SVM	Accuracy (Validation): 65.5%
Last change: Removed 5 features 1/6 features	
☆ 8 SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features	

Model 1

Model 2

Model 3

Model 4

Model 5

Model 6

Summary

Validation Confusion Matrix

Model 6: SVM

Status: Trained

Training Results

Accuracy (Validation) 65.3%

Total cost (Validation) 208

Prediction speed ~5600 obs/sec

Training time 6.6936 sec

Model size (Compact) ~14 kB

Model Hyperparameters

Feature Selection: 1/6 individual features selected

PCA: Disabled

Misclassification Costs: Default

Optimizer: Not applicable

۵- شاخص توده بدنی

Models

Sort by Model Number

1 Tree

Accuracy (Validation): 72.3%

Last change: Fine Tree 6/6 features

2 SVM

Accuracy (Validation): 77.3%

Last change: Linear SVM 6/6 features

3 SVM

Accuracy (Validation): 74.3%

Last change: Removed 5 features 1/6 features

4 SVM

Accuracy (Validation): 65.3%

Last change: Removed 5 features 1/6 features

5 SVM

Accuracy (Validation): 65.3%

Last change: Removed 5 features 1/6 features

6 SVM

Accuracy (Validation): 65.3%

Last change: Removed 5 features 1/6 features

7 SVM

Accuracy (Validation): 65.5%

Last change: Removed 5 features 1/6 features

8 SVM

Accuracy (Validation): 65.3%

Last change: Removed 5 features 1/6 features

Model 1 Model 2 Model 3 Model 4 Mod

Summary x Validation Confusion Matrix x

Model 7: SVM

Status: Trained

Training Results

Accuracy (Validation) 65.5%

Total cost (Validation) 207

Prediction speed ~10000 obs/sec

Training time 23.058 sec

Model size (Compact) ~14 kB

Model Hyperparameters

Feature Selection: 1/6 individual features selected

PCA: Disabled

Misclassification Costs: Default

Optimizer: Not applicable

Models

Sort by Model Number

1	Tree	Accuracy (Validation): 72.3%
Last change: Fine Tree 6/6 features		
2	SVM	Accuracy (Validation): 77.3%
Last change: Linear SVM 6/6 features		
3	SVM	Accuracy (Validation): 74.3%
Last change: Removed 5 features 1/6 features		
4	SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features		
5	SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features		
6	SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features		
7	SVM	Accuracy (Validation): 65.5%
Last change: Removed 5 features 1/6 features		
8	SVM	Accuracy (Validation): 65.3%
Last change: Removed 5 features 1/6 features		

Model 1 Model 2 Model 3 Model 4 M

Summary × Validation Confusion Matrix ×

Model 8: SVM
 Status: Trained

Training Results
 Accuracy (Validation) 65.3%
 Total cost (Validation) 208
 Prediction speed ~16000 obs/sec
 Training time 2.442 sec
 Model size (Compact) ~14 kB

▶ **Model Hyperparameters**
 ▶ **Feature Selection: 1/6 individual features selected**
 ▶ **PCA: Disabled**
 ▶ **Misclassification Costs: Default**
 ▶ **Optimizer: Not applicable**

نتیجه:

به ترتیب گلوکز و بعد شاخص توده بدنی بیشترین ربط روی دیابتی بودن فرد را دارد.

تمرین ۳-۳)

ماشین آموزش دیده را باتوجه به مراحل اکسپورت میکنیم و خودمان دقت را تخمین میزنیم:

```
%% p3 - 3
dataset = readtable('diabetes-training.csv');
labels = dataset(:, end);
features = dataset(:, 1 : end-1);
predictions = TrainedModel.predictFcn(features);
accuracy = mean(predictions == labels).*100;
disp(accuracy);
```

۱- ابتدا تمام دیتا ها را میخوانیم و ذخیره میکنیم.

۲- آخرین ستون را جدا میکنیم.

۳- تمامی ستونهای بهجز ستون آخر را که نشاندهنده ویژگیهای ورودی مدل هستند، جدا میکنیم.

۴- مدل آموزش دیده برای پیش بینی داده های ورودی استفاده میشود.

۵- دقت (فاز آموزش) را محاسبه میکنیم.

خروجی:

دقت گزارش شده، تطابق دارد.

```
Command Window

label
_____

77.5

fx >>
```

تمرین ۳-۴)

```
% p3 - 4
dataset = readtable('diabetes-validation.csv');
labels = dataset(:, end);
features = dataset(:, 1 : end-1);
predictions = TrainedModel.predictFcn(features);
accuracy = mean(predictions == labels).*100;
disp(accuracy);
```

دقت فاز تست یا ارزیابی:



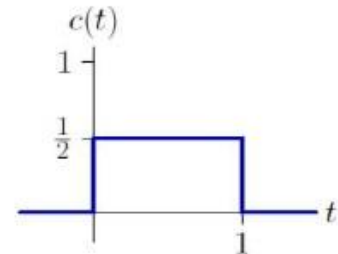
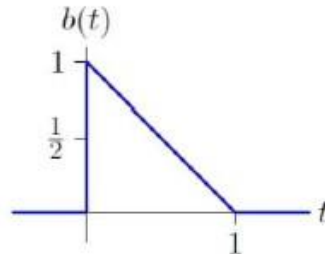
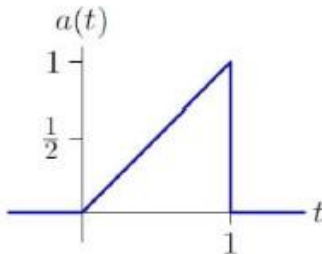
Command Window

label
78

fx >>

بخش چهارم:

کانولوشن دو سیگنال از سیگنال های زیر را در نظر بگیرید:



میخواهیم تعیین کنیم کدام یک از شکل ها میتواند از کانوالو شدن بقیه بدست بیاید و اگر این امکان پذیر است، نشان دهیم کدام یک از شکل ها در این ترکیب بوده اند. (*) در غیر این صورت ضربدر بگذاریم.

دقت کنید که در این سوال **به پاسخ ممکن** وجود دارد:

$$(a * a). (a * b). (a * c). (b * a). (b * b). (b * c). (c * a). (c * b). (c * c) \text{ or } (X * X)$$

هم چنین دقت کنید که جواب ممکن است **یکتا نباشد**. با استفاده از دستور `conv` در نرم افزار `MATLAB` به راحتی این تمرین را انجام دهید.

ابتدا شکل های سوال را رسم میکنیم که دوتا خط با شیب + و - است و یکی ثابت:

```
t = linspace(-10, 10, 1000);  
a = (t >= 0 & t < 1) .* t;  
b = (t >= 0 & t < 1) .* (1 - t);  
c = (t >= 0 & t <= 1)/2;
```

حالا باید فیگورها را جداگانه رسم کنیم:

```
signals = {'a', 'b', 'c'};  
data = {a, b, c};  
figure;  
idx = 1;  
for i = 1:3  
    subplot(1, 3, i);  
    xlim([-1, 3]);  
    plot(t, data{i}, 'LineWidth', 1);
```

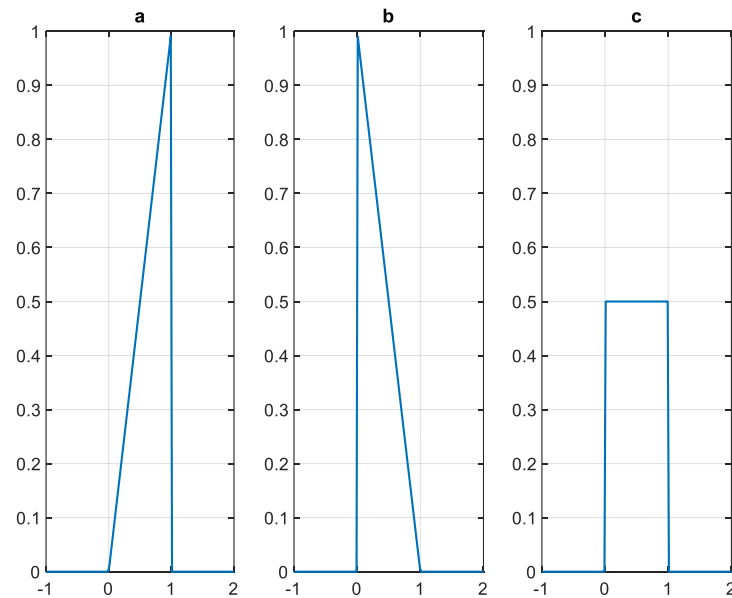
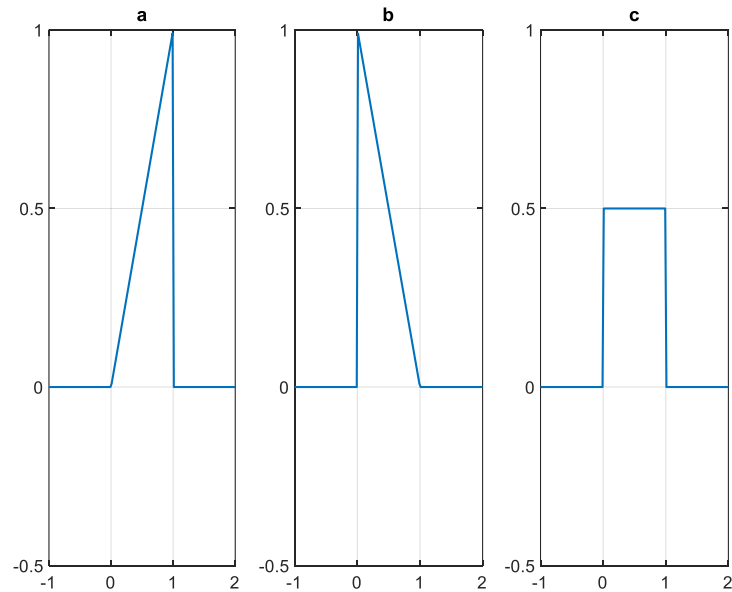
```
title(signals{i});
grid on;
xlim([-1, 2]);
ylim([-0.5,1])
```

این خط را میتوان یا به این صورت یعنی با محور عمودی ۰/۵ تا ۱ یا ۰/۱ نشان داد

```
ylim([0,1])
```

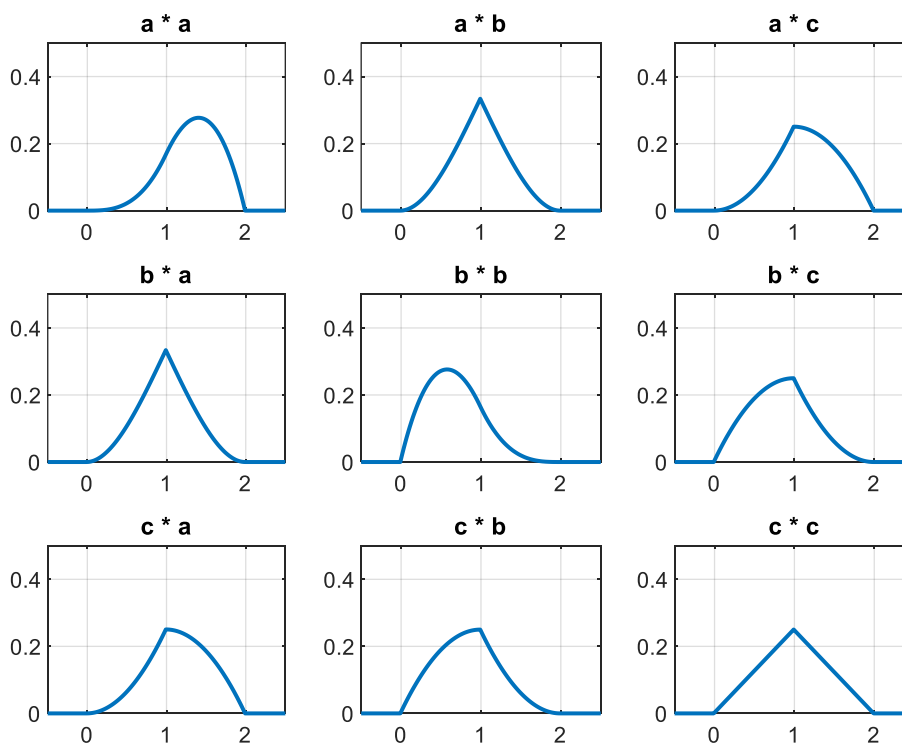
```
end
```

خروجی به صورت:

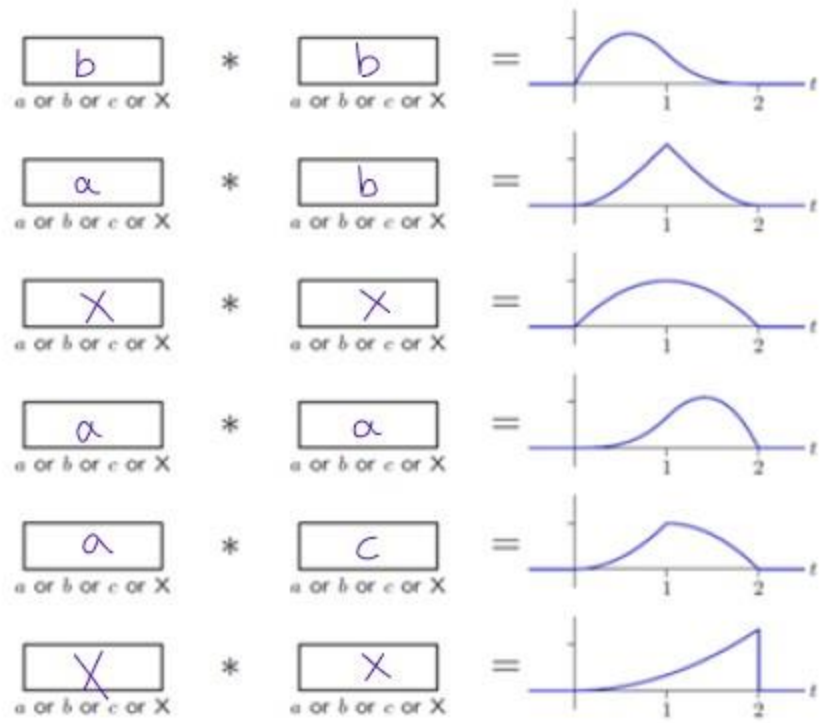


حالت های مختلف را با هم کانوالو میکنیم و رسم میکنیم تا ببینیم چند تا حالت بدست می آید و کدام شکل ها ساخته شده است. تعداد نمودار ها از تعداد کانولوشن ها بدست می آید. (در یک حلقه تو در تو)

```
%%
figure;
n = 1;
for i = 1:3
    for j = 1:3
        conv_result = conv(data{i}, data{j}, 'same') * (t(2) - t(1));
        subplot(3, 3, n);
        plot(t, conv_result, 'LineWidth', 1.5);
        title([signals{i}, ' * ', signals{j}]);
        xlim([-0.5, 2.5]);
        ylim([0, 0.5]);
        grid on;
        n = n + 1;
    end
end
end
```



میدانیم بعلت خاصیت جمعی بودن (سیگما یا انتگرال) کانولوشن، خاصیت جا به جایی دارد.



پایان